INNOPOLIS UNIVERSITY

SNE MASTER TRACK

2025

COURSE: [F25] Unix-based Internet Applications

CIA LAB 6 Assignment

On

Web Servers and Directory Service

Student Name: HASIB AL TAHSIN

Application ID: 187623

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 12.12.2025

Task 1 - Install & Configure Virtual Hosts

1. Download, verify, build and install the webserver daemon from the source. Note: Some features of your web server may be built-in or modularized. Enable at least SSL/TLS during your installation.

Answer:

I have chosen Apache web server for lab from the mentioned web servers.

Download, verify, build, install Apache from source

After completing apt update in terminal:

```
hasib@ns1:~$ sudo apt update
[sudo] password for hasib:
Sorry, try again.
[sudo] password for hasib:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,620 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main Icons (48x48) [36.0 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main Icons (64x64) [51.0 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [15.7 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,500 kB]
```

Install build tools and dependencies:

```
hasib@ns1:~$ sudo apt install -y build-essential wget tar \
  libpcre3 libpcre3-dev \
  libssl-dev \
  zlib1g-dev
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
wget is already the newest version (1.21.4-1ubuntu4.1).
wget set to manually installed.
tar is already the newest version (1.35+dfsg-3build1).
tar set to manually installed.
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu bzip2 dpkg-dev fakeroot
  g++ g++-13 g++-13-x86-64-linux-gnu g++-x86-64-linux-gnu gcc gcc-13
  gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu libalgorithm-diff-perl
  libalgorithm-diff-xs-perl libalgorithm-merge-perl libasan8 libbinutils
  libcc1-0 libctf-nobfd0 libctf0 libdpkg-perl libfakeroot
  libfile-fcntllock-perl libgcc-13-dev libgprofng0 libhwasan0 libitm1 liblsan0
  libpcre16-3 libpcre32-3 libpcrecpp0v5 libquadmath0 libsframe1
  libstdc++-13-dev libtsan2 libubsan1 lto-disabled-list make
```

First, the system package list is updated using the *sudo apt update* command, so that new software or dependencies can be installed. Then, the build-essential package is used to install all kinds of software compilation tools including *gcc, g++,* make. *wget* is used to download Apache source code from the internet, and tar is used to unzip or extract the *.tar.gz* archive file. Apache requires *libssl-dev* for SSL/TLS support. In addition, Apache modules (especially *mod_rewrite*) work on

the PCRE library, so *libpcre3* and *libpcre3-dev* are installed. *zlib1g-dev* is used here to support Apache's compression module.

Apache HTTPD source download

The */usr/local/src* directory was entered to download the Apache HTTP Server source, as this is generally used as the standard location for storing source code.

```
hasib@ns1:~$ cd /usr/local/src
```

After entering that folder using the *cd /usr/local/src* command, the latest version of Apache HTTPD, *httpd-2.4.65.tar.gz* file, was downloaded using the *wget* command.

```
hasib@ns1:/usr/local/src$ sudo wget https://downloads.apache.org/httpd/httpd-2.4.65.tar.gz
[sudo] password for hasib:
--2025-11-29 00:40:57--  https://downloads.apache.org/httpd/httpd-2.4.65.tar.gz
Resolving downloads.apache.org (downloads.apache.org)... 135.181.214.104, 88.99.208.237, 2a01:4f9:3a:2c57::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|135.181.214.104|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 9814591 (9.4M) [application/x-gzip]
Saving to: ‘httpd-2.4.65.tar.gz’

httpd-2.4.65.tar.gz 100%[===================>]   9.36M  15.6MB/s    in 0.6s

2025-11-29 00:40:59 (15.6 MB/s) - ‘httpd-2.4.65.tar.gz’ saved [9814591/9814591]
```

File Verify - Simple Checksum

After downloading the Apache HTTPD source file, first we had to make sure that the file was actually downloaded correctly. For this, we used the ls command to check whether the file was in the directory or not. Then we ran the *sha256sum* command to extract the SHA256 checksum of that file. This checksum was compared with the checksum given on Apache's official download page, so that it can be understood that the downloaded file is original or intact.

```
hasib@ns1:/usr/local/src$ ls
httpd-2.4.65.tar.gz
```

Apache HTTPD 2.4.65 version has been successfully downloaded and is correctly in the folder.

View SHA256 checksum:

```
hasib@ns1:/usr/local/src$ sha256sum httpd-2.4.65.tar.gz
4f92861a50325c6d1046ebad5d814bff0d4169ada8cc265655f32b7f1ba4be1b  httpd-2.4.65.tar.gz
```

The output is a long 64-character hexadecimal SHA256 hash. This output indicates that the file is not corrupt and the download is complete.

Unzip the source

Unzip the httpd source:

```
hasib@ns1:/usr/local/src$ cd httpd-2.4.65
```

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ /usr/local/src/httpd-2.4.65
bash: /usr/local/src/httpd-2.4.65: Is a directory
```

```
hasib@ns1:/usr/local/src$ cd /usr/local/src/httpd-2.4.65
hasib@ns1:/usr/local/src/httpd-2.4.65$ ls
ls srclib
ABOUT_APACHE     CMakeLists.txt  include         NWGNUmakefile
acinclude.m4     config.layout   INSTALL         os
Apache-apr2.dsw  config.log      InstallBin.dsp  README
Apache.dsw       config.nice     LAYOUT          README.CHANGES
apache_probes.d  configure       libhttpd.dep    README.cmake
ap.d             configure.in    libhttpd.dsp    README.platforms
build            docs            libhttpd.mak    ROADMAP
BuildAll.dsp     emacs-style     LICENSE         server
BuildBin.dsp     httpd.dep       Makefile.in     srclib
buildconf        httpd.dsp       Makefile.win    support
CHANGES          httpd.mak       modules         test
changes-entries  httpd.spec      NOTICE          VERSIONING
Makefile.in
```

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo make clean
make: *** No rule to make target 'clean'.  Stop.
```

Configure - Enable SSL/TLS and http2

Then Apache is configured and built and SSL is enabled:

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo ./configure \
  --prefix=/usr/local/apache2 \
  --enable-so \
  --enable-ssl \
  --with-ssl=/usr \
  --enable-rewrite
checking for chosen layout... Apache
checking for working mkdir -p... yes
checking for grep that handles long lines and -e... /usr/bin/grep
checking for egrep... /usr/bin/grep -E
checking build system type... x86_64-pc-linux-gnu
checking host system type... x86_64-pc-linux-gnu
checking target system type... x86_64-pc-linux-gnu
configure:
configure: Configuring Apache Portable Runtime library...
configure:
checking for APR... yes
  setting CC to "x86_64-linux-gnu-gcc"
  setting CPP to "x86_64-linux-gnu-gcc -E"
  setting CFLAGS to " "
  setting CPPFLAGS to "  -DLINUX -D_REENTRANT -D_GNU_SOURCE"
  setting LDFLAGS to " "
configure:
configure: Configuring Apache Portable Runtime Utility library...
configure:
checking for APR-util... yes
checking for gcc... x86_64-linux-gnu-gcc
checking whether the C compiler works... yes
checking for C compiler default output file name... a.out
checking for suffix of executables...
checking whether we are cross compiling... no
```

```
config.status: creating include/ap_config_auto.h
config.status: executing default commands
configure: summary of build options:

    Server Version: 2.4.65
    Install prefix: /usr/local/apache2
    C compiler:     x86_64-linux-gnu-gcc
    CFLAGS:
    CPPFLAGS:         -DLINUX -D_REENTRANT -D_GNU_SOURCE
    LDFLAGS:
    LIBS:
    C preprocessor: x86_64-linux-gnu-gcc -E
```

Apache installation uses various configuration flags that control the capabilities and features of the server. For example, *--prefix=/usr/local/apache2* will install Apache in that specific folder, so you know exactly where the files are. *--enable-so* enables dynamic module support, which means you can load or unload modules later as needed-making server management much more flexible. *--enable-ssl* and *--with-ssl=/usr* enable SSL/TLS support on the server, which is mandatory for running HTTPS and is also very important from a security perspective. Most modern web applications use the *--enable-rewrite* flag, because it allows URL Rewriting and is very useful for practical tasks, such as creating pretty URLs or configuring redirects. Finally, *--enable-http2* enables HTTP/2 support, which ensures faster loading, multiplexing, and improved performance. Together, these make Apache a more powerful, secure, and modern web standards-compliant server.

Build & Install

Build

```
    C preprocessor: x86_64-linux-gnu-gcc -E

hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo make
Making all in srclib
make[1]: Entering directory '/usr/local/src/httpd-2.4.65/srclib'
make[1]: Leaving directory '/usr/local/src/httpd-2.4.65/srclib'
Making all in os
make[1]: Entering directory '/usr/local/src/httpd-2.4.65/os'
Making all in unix
make[2]: Entering directory '/usr/local/src/httpd-2.4.65/os/unix'
make[3]: Entering directory '/usr/local/src/httpd-2.4.65/os/unix'
make[3]: Nothing to be done for 'local-all'.
make[3]: Leaving directory '/usr/local/src/httpd-2.4.65/os/unix'
make[2]: Leaving directory '/usr/local/src/httpd-2.4.65/os/unix'
make[1]: Leaving directory '/usr/local/src/httpd-2.4.65/os'
Making all in server
make[1]: Entering directory '/usr/local/src/httpd-2.4.65/server'
Making all in mpm
make[2]: Entering directory '/usr/local/src/httpd-2.4.65/server/mpm'
Making all in event
make[3]: Entering directory '/usr/local/src/httpd-2.4.65/server/mpm/event'
make[4]: Entering directory '/usr/local/src/httpd-2.4.65/server/mpm/event'
make[4]: Nothing to be done for 'local-all'.
make[4]: Leaving directory '/usr/local/src/httpd-2.4.65/server/mpm/event'
make[3]: Leaving directory '/usr/local/src/httpd-2.4.65/server/mpm/event'
make[2]: Leaving directory '/usr/local/src/httpd-2.4.65/server/mpm'
make[2]: Entering directory '/usr/local/src/httpd-2.4.65/server'
Found newer headers. Will rebuild exports.c.
rm -f exports.c export_files
```

Installing:

```
make[2]: Entering directory '/usr/local/src/httpd-2.4.65/support'
make[2]: Leaving directory '/usr/local/src/httpd-2.4.65/support'

make[1]: Leaving directory '/usr/local/src/httpd-2.4.65'
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo make install
Making install in srclib
make[1]: Entering directory '/usr/local/src/httpd-2.4.65/srclib'
make[2]: Entering directory '/usr/local/src/httpd-2.4.65/srclib'
make[2]: Leaving directory '/usr/local/src/httpd-2.4.65/srclib'
make[1]: Leaving directory '/usr/local/src/httpd-2.4.65/srclib'
Making install in os
make[1]: Entering directory '/usr/local/src/httpd-2.4.65/os'
Making install in unix
make[2]: Entering directory '/usr/local/src/httpd-2.4.65/os/unix'
make[3]: Entering directory '/usr/local/src/httpd-2.4.65/os/unix'
make[3]: Leaving directory '/usr/local/src/httpd-2.4.65/os/unix'
make[2]: Leaving directory '/usr/local/src/httpd-2.4.65/os/unix'
make[2]: Entering directory '/usr/local/src/httpd-2.4.65/os'
make[2]: Leaving directory '/usr/local/src/httpd-2.4.65/os'
make[1]: Leaving directory '/usr/local/src/httpd-2.4.65/os'
Making install in server
make[1]: Entering directory '/usr/local/src/httpd-2.4.65/server'
Making install in mpm
make[2]: Entering directory '/usr/local/src/httpd-2.4.65/server/mpm'
Making install in event
make[3]: Entering directory '/usr/local/src/httpd-2.4.65/server/mpm/event'
make[4]: Entering directory '/usr/local/src/httpd-2.4.65/server/mpm/event'
```

Checking if the installation was successful

After the entire Apache installation process is complete, the first thing to do is to run the *ls /usr/local/apache2/bin* command to see if the main Apache binaries - such as *apachectl, httpd*, etc. - have been created correctly to understand that the build and installation have been completed successfully. Here, *apachectl, httpd*, etc. are visible. So, the build and installation were OK.

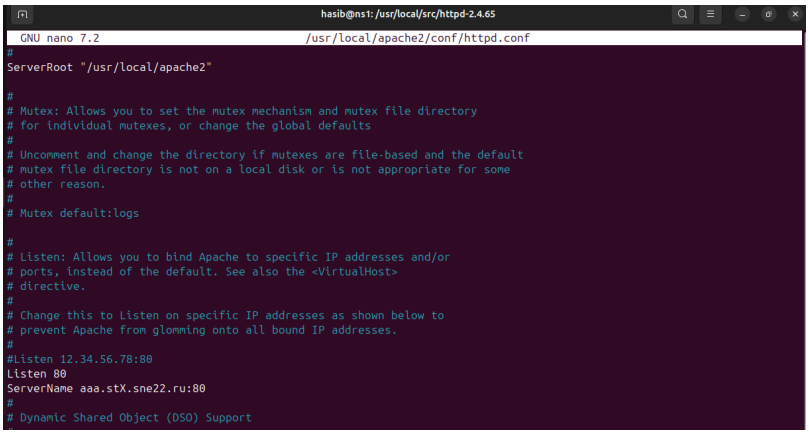
```
Installing man pages and online manual
make[1]: Leaving directory '/usr/local/src/httpd-2.4.65'
hasib@ns1:/usr/local/src/httpd-2.4.65$ ls /usr/local/apache2/bin
ab         checkgid   envvars-std   htdbm     httpd       rotatelogs
apachectl  dbmmanage  fcgistarter   htdigest  httxt2dbm
apxs       envvars    htcacheclean  htpasswd  logresolve
```

Then comes the configuration part, where you have to open the *httpd.conf* file and adjust some important settings. Put the Listen 80 line to instruct the server to bind to port 80, add a *ServerName* to tell Apache its identity, and create two separates <*VirtualHost*> blocks as needed, such as *aaa* and *bbb* - where the domain, document root, and other settings are specified for each virtual site. As a result, multiple websites or apps can be managed from one server. SSL module is loaded, that's already done:

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo nano /usr/local/apache2/conf/httpd.conf
hasib@ns1:/usr/local/src/httpd-2.4.65$ /usr/local/apache2/bin/httpd -M | grep ssl
 ssl_module (shared)
```

Once installed, after loading Apache:

The command */usr/local/apache2/bin/httpd -M | grep ssl* was run to ensure that the SSL module was loaded correctly. If the output shows *ssl_module (shared*), SSL/TLS support is enabled and there are no problems configuring HTTPS later.



Config syntax test:

After changing the Apache configuration, you must check whether the syntax is correct or not with the following command. If it shows "*Syntax OK*".

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ /usr/local/apache2/bin/apachectl configtest
Syntax OK
```

soft reload (no downtime):

Graceful reload is given in the next step so that even though the service is reloaded, there is no downtime. For this, the following command is used.

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo /usr/local/apache2/bin/apachectl graceful
httpd not running, trying to start
```

This takes some time.

Check:

To verify that Apache processes are actually running, I ran the following command and saw a few httpd processes. This shows that the server was fully up and running and ready to accept requests.

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ ps aux | grep httpd
root       70177  0.0  0.0  12756  6564 ?        Ss   01:21   0:00 /usr/local/apache2/bin/httpd -k graceful
daemon     70178  0.0  0.0 1939816 6840 ?        Sl   01:21   0:00 /usr/local/apache2/bin/httpd -k graceful
daemon     70179  0.0  0.0 2005352 6844 ?        Sl   01:21   0:00 /usr/local/apache2/bin/httpd -k graceful
daemon     70180  0.0  0.0 1939816 6844 ?        Sl   01:21   0:00 /usr/local/apache2/bin/httpd -k graceful
hasib      70269  0.0  0.0   9144  2272 pts/0    S+   01:21   0:00 grep --color=auto httpd
```

After the installation is complete, Apache's bin directory is visible:

The following command has been used to confirm that the required commands are present correctly.

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ ls /usr/local/apache2/bin
ab         checkgid   envvars-std   htdbm     httpd       rotatelogs
apachectl  dbmmanage  fcgistarter   htdigest  httxt2dbm
apxs       envvars    htcacheclean  htpasswd  logresolve
```

If *apachectl*, httpd, etc. are there, you will know it is working properly.

2. Define the root directory and then two virtual hosts (and configure DNS records or wildcard accordingly):

aaa.stX.sne22.ru

bbb.stX.sne22.ru

Answer:

Setting Root Directory + Virtual Hosts

After Apache installation, I selected a root directory to host the website and then configured multiple VirtualHosts. Here, two subdomains are used:

- **aaa.stX.sne22.ru**
- **bbb.stX.sne22.ru**

Here stX is your group/ID-like st3, st5 etc. Through this, two different websites or projects can be hosted on the same server.

Which directory as Document Root?

Apache default config is in */usr/local/apache2/conf/httpd.conf.*

I used */var/www as DocumentRoot* of virtual host. This command creates a directory for the new website *aaa.stX.sne22.ru*. Since there is *-p*, it creates it even if the parent folder does not exist.

Create the directory:

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo mkdir -p /var/www/aaa.stX.sne22.ru
```

Similarly create a directory for the second website *bbb.stX.sne22.ru*:

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo mkdir -p /var/www/bbb.stX.sne22.ru
```

Giveing ownership of the */var/www folder* to the user, so that files can be easily edited. *-R* means recursive, changes ownership of all files/folders.

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo chown -R $USER:$USER /var/www
```

Enabling Name-based VirtualHost

Open the Apache config file:

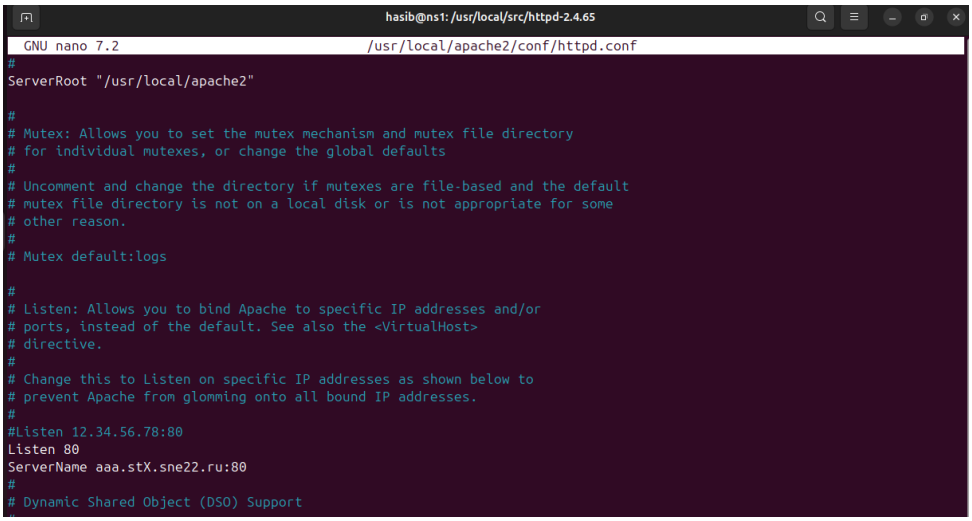
```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo nano /usr/local/apache2/conf/httpd.conf
```

Setting Listen and ServerName

Telling Apache to listen for incoming requests on HTTP port 80. So, Listen 80

is added there. Then a main *ServerName* is set to tell Apache what its own primary hostname should be.

We will give a separate ServerName for *VirtualHost* later. Only a main *ServerName* is needed, the rest of the work is done in *VirtualHost.*



Configuring the *VirtualHost* Block

The following block is added at the very bottom of the *httpd.conf* file:



```
<VirtualHost *:80>
    ServerName bbb.stX.sne22.ru
    ServerAlias www.bbb.stX.sne22.ru
    DocumentRoot "/var/www/bbb.stX.sne22.ru"

    <Directory "/var/www/bbb.stX.sne22.ru">
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog "/usr/local/apache2/logs/bbb_error.log"
    CustomLog "/usr/local/apache2/logs/bbb_access.log" common
</VirtualHost>


^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C L
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ G
```

The *<VirtualHost *:80>* line tells Apache that all the settings in this block apply to requests coming from any IP on port 80. Then, the main domain of the *VirtualHost* is set to *ServerName aaa.stX.sne22.ru*, and the *ServerAlias www.aaa.stX.sne22.ru* ensures that the redirection to the same site with www is ensured. The *DocumentRoot "/var/www/aaa.stX.sne22.ru*" specifies the directory from which the main files of the site will be served. Inside the directory block, the directory listing and symbolic link following feature is enabled using Options Indexes *FollowSymLinks*, and additional configuration is allowed via the *.htaccess* file with *AllowOverride* All. Adding Require all granted allows access to all clients on the site. Finally, *ErrorLog* and *CustomLog* are specified so that the error and access records of this site are stored in separate log files. The block ends with *</VirtualHost>,* which completes the *VirtualHost* configuration for *aaa.stX.sne22.ru.*

3. Create a simple, unique HTML page for each virtual host to make sure that the server can correctly serve it. Testing:

Answer:

Creating beautiful HTML pages for each Virtual Host

Index.html of aaa.stX.sne22.ru

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ nano /var/www/aaa.stX.sne22.ru/index.html
```

To verify that each *VirtualHost* is working properly, two separate HTML pages have been created (one for *aaa.stX.sne22.ru* and one for *bbb.stX.sne22.ru*). Although the instructions call for simple HTML, both pages have been designed to be a bit more beautiful, informative, and visually attractive, so that it is clear during testing which site is being served from which *VirtualHost*.

```
 GNU nano 7.2                                /var/www/aaa.stX.sne22.ru/index.html
<!DOCTYPE html>
<html lang="bn">
<head>
    <meta charset="UTF-8">
    <title>AAA Site - stX<</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background: linear-gradient(135deg, #0f172a, #1d4ed8);
            color: #f9fafb;
            margin: 0;
            padding: 0;
        }
        header {
            padding: 20px;
            text-align: center;
            background: rgba(15, 23, 42, 0.8);
        }
        header h1 {
            margin: 0;
            font-size: 2.2rem;
        }
        main {
            max-width: 900px;
            margin: 30px auto;
            background: rgba(15, 23, 42, 0.85);
            padding: 20px 30px;
            border-radius: 12px;
```

First, an index.html file is created inside the *DocumentRoot /var/www/aaa.stX.sne22.ru* of *aaa.stX.sne22.ru*. This page contains a summary of the Apache *VirtualHost* configuration, server info, HTTP test instructions, and a completely different design to make it easily recognizable. After adding the HTML code, the file is saved, so now when I check it using a browser or *curl,* it is clear that the *AAA VirtualHost* is loading correctly.

```
 GNU nano 7.2                                /var/www/aaa.stX.sne22.ru/index.html
            box-shadow: 0 10px 30px rgba(0,0,0,0.4);
        }
        h2 {
            color: #a5b4fc;
        }
        .info-box {
            border-left: 4px solid #22c55e;
            padding: 10px 15px;
            background: rgba(15, 118, 110, 0.3);
            margin: 15px 0;
            border-radius: 8px;
        }
        code {
            background: rgba(15,23,42,0.8);
            padding: 3px 6px;
            border-radius: 4px;
            font-size: 0.9rem;
        }
        footer {
            text-align: center;
            padding: 15px;
            font-size: 0.9rem;
            opacity: 0.7;
        }
        a {
            color: #38bdf8;
            text-decoration: none;
        }
```

```
 GNU nano 7.2                                  /var/www/aaa.stX.sne22.ru/index.html
        }
        a {
            color: #38bdf8;
            text-decoration: none;
        }
        a:hover {
            text-decoration: underline;
        }
    </style>
</head>
<body>
<header>
    <h1>AAA Virtual Host - stX</h1>
    <p>Apache Web Server Lab - CIA Lab 6</p>
</header>
Floppy Disk
    <h2> What is this page for? </h2>
    <p>
        This HTML page was created for the domain <strong>aaa.stX.sne22.ru</strong>.
This proves that your Apache server is working correctly
as a <em>Name-based Virtual Host</em>.
    </p>

    <div class="info-box">
        <h3>Server Information</h3>
        <ul>
            <li>Web Server: Apache HTTPD (compiled from source)</li>
```

```
 GNU nano 7.2                          /var/www/aaa.stX.sne22.ru/index.html
        <h3>Server Information</h3>
        <ul>
            <li>Web Server: Apache HTTPD (compiled from source)</li>
            <li>Virtual Host: aaa.stX.sne22.ru</li>
            <li>Document Root: <code>/var/www/aaa.stX.sne22.ru</code></li>
            <li>Lab: CIA Lab 6 - Web Servers and Directory Service</li>
        </ul>
    </div>

    <h2>Instructions for testing HTTP/1.1</h2>
    <ol>
        <li>Open the terminal from the client machine.</li>
        <li>Enter the command: <code>curl -v http://aaa.stX.sne22.ru/</code></li>
        <li>You can see the full HTTP request and response headers.</li>
    </ol>

    <p>
        If you look at this page, you will understand that:
        <strong>Virtual Host configuration is working successfully!</strong>
    </p>
</main>

<footer>
    &copy; 2025 - AAA Virtual Host (stX) | Designed for CIA Lab 6
</footer>
</body>
</html>

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy
```

index.html of bbb.stX.sne22.ru

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ nano /var/www/bbb.stX.sne22.ru/index.html
```

Another index.html has also been created in the *DocumentRoot /var/www/bbb.stX.sne22.ru* of the second *VirtualHost bbb.stX.sne22.ru*. This page has been created in a different color, layout, and presentation than *AAA*, so that the two sites can be easily distinguished from each other. The page provides a *VirtualHost* config summary, notes, domain-root information, and a guide to viewing HTTP requests with curl. This provides a very clear understanding of how multiple site hosting works.

```
 GNU nano 7.2                          /var/www/bbb.stX.sne22.ru/index.html
<!DOCTYPE html>
<html lang="bn">
<head>
    <meta charset="UTF-8">
    <title>BBB Site - stX</title>
    <style>
        body {
            font-family: "Segoe UI", Tahoma, sans-serif;
            background: radial-gradient(circle at top, #dc2626, #111827);
            color: #f9fafb;
            margin: 0;
            padding: 0;
        }
        header {
            padding: 25px;
            text-align: center;
            background: rgba(17,24,39,0.85);
        }
        header h1 {
            margin: 0;
            font-size: 2.4rem;
        }
        main {
            max-width: 900px;
            margin: 30px auto;
            background: rgba(31,41,55,0.95);
            padding: 25px 35px;
            border-radius: 14px;

^G Help       ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit       ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

These two HTML pages have successfully tested Apache's Name-based *VirtualHost* configuration. Now, typing different domains in the browser displays the content of the corresponding *VirtualHost,* confirming that the *VirtualHost* setup and directory structure are created correctly.

```
 GNU nano 7.2                          /var/www/bbb.stX.sne22.ru/index.html
            text-align: left;
            font-size: 0.95rem;
        }
        th {
            background: #374151;
        }
        code {
            background: rgba(15,23,42,0.9);
            padding: 3px 6px;
            border-radius: 4px;
        }
        footer {
            text-align: center;
            padding: 15px;
            font-size: 0.9rem;
            opacity: 0.75;
        }
    </style>
</head>
<body>
<header>
    <h1>BBB Virtual Host - stX</h1>
    <p><span class="badge">Secondary Host</span>   Apache Web Server Lab</p>
</header>

<main>
    <h2>The purpose of this host</h2>
    <p>
```

```
 GNU nano 7.2                          /var/www/bbb.stX.sne22.ru/index.html
<main>
    <h2>The purpose of this host</h2>
    <p>
        This page is created for the domain <strong>bbb.stX.sne22.ru</strong>.
        It is acting as a second Virtual Host so that you can understand the concept of multiple site hosting.
    </p>

    <div class="note">
        <strong>Note:</strong> AAA and BBB - although these two are separate hosts,
        they are being served from the same Apache server,
        but with different <em>DocumentRoot</em> and different domain names.
    </div>

    <h2>Configuration summary</h2>
    <table>
        <tr>
            <th>Property</th>
            <th>Value</th>
        </tr>
        <tr>
            <td>ServerName</td>
            <td>bbb.stX.sne22.ru</td>
        </tr>
        <tr>
            <td>DocumentRoot</td>
            <td><code>/var/www/bbb.stX.sne22.ru</code></td>
        </tr>
        <tr>
```

```
 GNU nano 7.2                          /var/www/bbb.stX.sne22.ru/index.html
        </tr>
        <tr>
            <td>ErrorLog</td>
            <td><code>/usr/local/apache2/logs/bbb_error.log</code></td>
        </tr>
        <tr>
            <td>AccessLog</td>
            <td><code>/usr/local/apache2/logs/bbb_access.log</code></td>
        </tr>
    </table>

    <h2>How to view HTTP/1.1 sessions with curl</h2>
    <ol>
        <li>Open the Terminal.</li>
        <li>Run the following command:
            <br>
            <code>curl -v http://bbb.stX.sne22.ru/</code>
        </li>
        <li>Note the request & response headers and explain them in the report.</li>
    </ol>
</main>

<footer>
    BBB Host (stX) - Designed to test Apache Virtual Hosts | CIA Lab 6
</footer>
</body>
</html>


^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy
```

4. Check the configuration syntax, start the daemon and enable it at boot time.

Answer:

Config Syntax Test

To check if there are any errors in the configuration file. For this, the *apachectl configtest* command is run, which scans the entire *httpd.conf* and *VirtualHost* blocks and checks if the syntax is correct.

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo /usr/local/apache2/bin/apachectl configtest
[sudo] password for hasib:
Syntax OK
```

After the command was successfully run, the terminal displayed the message "*Syntax OK*", which confirms that there are no typos or incorrect instructions in the Apache configuration.

Starting the Apache Server

After confirming that the syntax is correct, the Apache server has been started. As soon as the HTTP daemon is started using the *apachectl* start command, multiple httpd processes have been created in the system.

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo /usr/local/apache2/bin/apachectl start
```

To check:

The processes were viewed by running the command *ps aux | grep httpd* and confirmed that the server is running successfully in the background. This proves that the *VirtualHost* configs including the main service are loaded correctly.

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ ps aux | grep httpd
root        5193  0.0  0.0  12780  6652 ?        Ss   02:11   0:00 /usr/local/apache2/bin/httpd -k start
daemon      5194  0.0  0.0 2005368 6712 ?        Sl   02:11   0:00 /usr/local/apache2/bin/httpd -k start
daemon      5195  0.0  0.0 2005368 6716 ?        Sl   02:11   0:00 /usr/local/apache2/bin/httpd -k start
daemon      5196  0.0  0.0 2005368 6716 ?        Sl   02:11   0:00 /usr/local/apache2/bin/httpd -k start
hasib       5285  0.0  0.0   9144  2272 pts/0    S+   02:11   0:00 grep --color=auto httpd
```

Test from Browser

In this step, DNS resolution has been set on the local machine to test the *VirtualHost* from the browser. The *ip a* command has been used to check the server IP.

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:ae:b1:fd brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.100.10/24 brd 192.168.100.255 scope global noprefixroute ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:feae:b1fd/64 scope link
       valid_lft forever preferred_lft forever
3: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:ae:b1:07 brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    inet 192.168.218.130/24 brd 192.168.218.255 scope global dynamic noprefixroute ens34
       valid_lft 1111sec preferred_lft 1111sec
    inet6 fe80::3a55:df58:97c6:86c0/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Then the domain mapping is added to the */etc/hosts* file, for example:

192.168.56.10 aaa.stX.sne22.ru

192.168.56.10 bbb.stX.sne22.ru

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo nano /etc/hosts
[sudo] password for hasib:
```

This tells the local computer that these domains point to that specific server IP.

```
hasib@ns1: /usr/local/src/httpd-2.4.65
  GNU nano 7.2                                        /etc/hosts
192.168.218.130   aaa.stX.sne22.ru
192.168.218.130   bbb.stX.sne22.ru

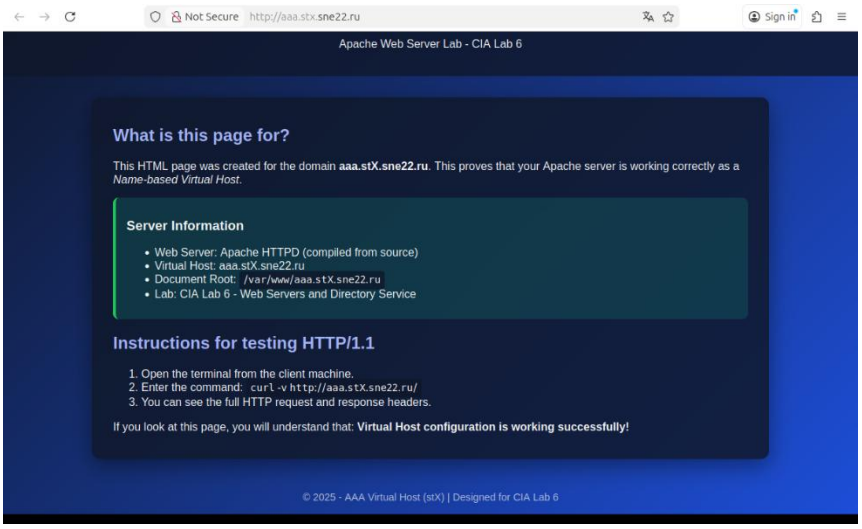
# The following lines are desirable for IPv6 capable hosts
::1     ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

I saved all and then exit.

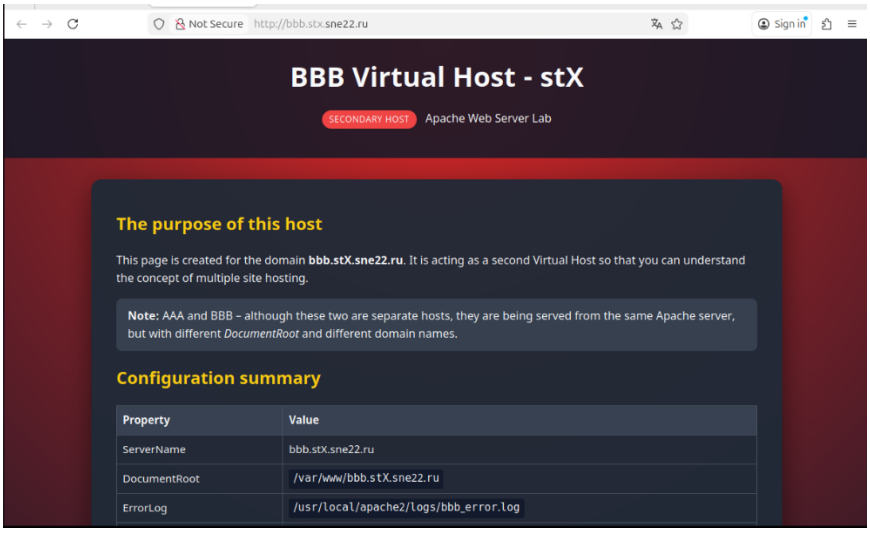
After saving the hosts file, visiting the domains from the browser loaded the corresponding HTML pages, confirming that both the *VirtualHost* configuration and the Apache server are working correctly.

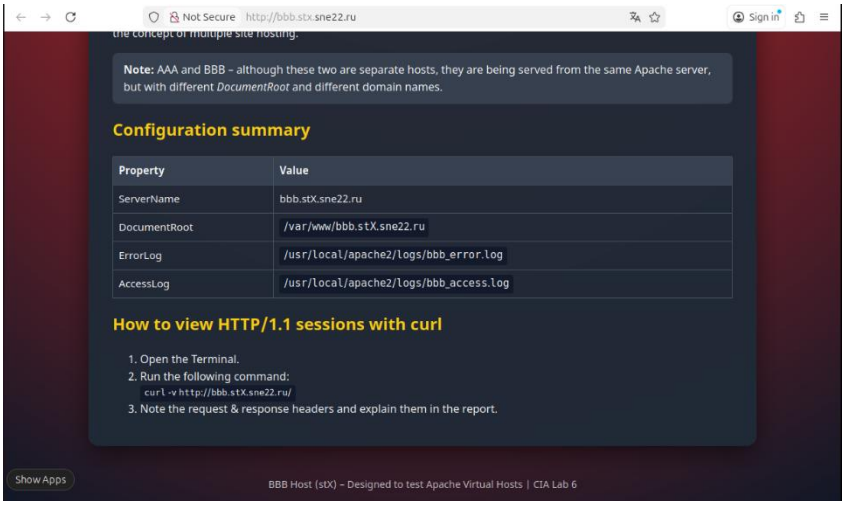
In the browser:

http://aaa.stX.sne22.ru - first page



http://bbb.stX.sne22.ru - second page





Now, by opening http://aaa.stX.sne22.ru and http://bbb.stX.sne22.ru in the browser, we can see that both Virtual Hosts are serving their own, separate HTML pages correctly.

For AAA host

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ nano /var/www/aaa.stX.sne22.ru/index.html
```

The */var/www/aaa.stX.sne22.ru/index.*html file for the *AAA* host has been created with a clean, modern design - gradient background, separate info box, server information, and instructions for testing *HTTP/1.1* with the *curl -v http://aaa.stX.sne22.ru/* command. This indicates that the *AAA* Virtual Host is serving content from the correct *DocumentRoot*, and the screenshots viewed in the browser visually confirm this configuration.

```
 GNU nano 7.2                          /var/www/aaa.stX.sne22.ru/index.html
<!DOCTYPE html>
<html lang="bn">
<head>
    <meta charset="UTF-8">
    <title>AAA Site - stX</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background: linear-gradient(135deg, #0f172a, #1d4ed8);
            color: #f9fafb;
            margin: 0;
            padding: 0;
        }
        header {
            padding: 20px;
            text-align: center;
            background: rgba(15, 23, 42, 0.8);
        }
        header h1 {
            margin: 0;
            font-size: 2.2rem;
        }
        main {
            max-width: 900px;
            margin: 30px auto;
            background: rgba(15, 23, 42, 0.85);
            padding: 20px 30px;
            border-radius: 12px;
```

```
 GNU nano 7.2                          /var/www/aaa.stX.sne22.ru/index.html
            box-shadow: 0 10px 30px rgba(0,0,0,0.4);
        }
        h2 {
            color: #a5b4fc;
        }
        .info-box {
            border-left: 4px solid #22c55e;
            padding: 10px 15px;
            background: rgba(15, 118, 110, 0.3);
            margin: 15px 0;
            border-radius: 8px;
        }
        code {
            background: rgba(15,23,42,0.8);
            padding: 3px 6px;
            border-radius: 4px;
            font-size: 0.9rem;
        }
        footer {
            text-align: center;
            padding: 15px;
            font-size: 0.9rem;
            opacity: 0.7;
        }
        a {
            color: #38bdf8;
            text-decoration: none;
        }
```

```
            text-decoration: underline;
        }
    </style>
</head>
<body>
<header>
    <h1>AAA Virtual Host - stX</h1>
    <p>Apache Web Server Lab - CIA Lab 6</p>
</header>

<main>
    <h2> What is this page for? </h2>
    <p>
        This HTML page was created for the domain <strong>aaa.stX.sne22.ru</strong>.
This proves that your Apache server is working correctly
as a <em>Name-based Virtual Host</em>.
    </p>

    <div class="info-box">
        <h3>Server Information</h3>
        <ul>
            <li>Web Server: Apache HTTPD (compiled from source)</li>
            <li>Virtual Host: aaa.stX.sne22.ru</li>
            <li>Document Root: <code>/var/www/aaa.stX.sne22.ru</code></li>
            <li>Lab: CIA Lab 6 - Web Servers and Directory Service</li>
        </ul>
    </div>
```

```
    </div>

    <h2>Instructions for testing HTTP/1.1</h2>
    <ol>
        <li>Open the terminal from the client machine.</li>
        <li>Enter the command: <code>curl -v http://aaa.stX.sne22.ru/</code></li>
        <li>You can see the full HTTP request and response headers.</li>
    </ol>

    <p>
        If you look at this page, you will understand that:
        <strong>Virtual Host configuration is working successfully!</strong>
    </p>
</main>

<footer>
    &copy; 2025 - AAA Virtual Host (stX) | Designed for CIA Lab 6
</footer>
</body>
</html>

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy
```

For BBB host

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ nano /var/www/bbb.stX.sne22.ru/index.html
```

Similarly, the */var/www/bbb.stX.sne22.ru/index.html* file has been created for the BBB host, which has been designed with a slightly different look and feel - a red-toned gradient background, a "Secondary Host" badge, and a table below that shows the configuration summary for *ServerName, DocumentRoot, ErrorLog,* and *AccessLog*. Also, a separate section shows how to view *HTTP/1.1* sessions by running the command *curl -v http://bbb.stX.sne22.ru/* . Both pages are gorgeous + informative, and clearly distinct from each other, making it easy to test which domain is serving which Virtual Host.

```
 GNU nano 7.2                                /var/www/bbb.stX.sne22.ru/index.html
<!DOCTYPE html>
<html lang="bn">
<head>
    <meta charset="UTF-8">
    <title>BBB Site - stX</title>
    <style>
        body {
            font-family: "Segoe UI", Tahoma, sans-serif;
            background: radial-gradient(circle at top, #dc2626, #111827);
            color: #f9fafb;
            margin: 0;
            padding: 0;
        }
        header {
            padding: 25px;
            text-align: center;
            background: rgba(17,24,39,0.85);
        }
        header h1 {
            margin: 0;
            font-size: 2.4rem;
        }
        main {
            max-width: 900px;
            margin: 30px auto;
            background: rgba(31,41,55,0.95);
            padding: 25px 35px;
            border-radius: 14px;

^G Help       ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit       ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

Show Apps

```
            box-shadow: 0 12px 35px rgba(0,0,0,0.5);
        }
        h2 {
            color: #facc15;
        }
        .note {
            background: rgba(55,65,81,0.9);
            padding: 12px 16px;
            border-radius: 8px;
            margin: 15px 0;
        }
        .badge {
            display: inline-block;
            padding: 4px 10px;
            border-radius: 999px;
            background: #ef4444;
            font-size: 0.75rem;
            text-transform: uppercase;
            letter-spacing: 0.05em;
        }
        table {
            width: 100%;
            border-collapse: collapse;
            margin-top: 15px;
        }
        th, td {
            border: 1px solid #4b5563;
            padding: 8px;

^G Help       ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set
^X Exit       ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

```
 GNU nano 7.2                                /var/www/bbb.stX.sne22.ru/index.html
        th, td {
            border: 1px solid #4b5563;
            padding: 8px;
            text-align: left;
            font-size: 0.95rem;
        }
        th {
            background: #374151;
        }
        code {
            background: rgba(15,23,42,0.9);
            padding: 3px 6px;
            border-radius: 4px;
        }
        footer {
            text-align: center;
            padding: 15px;
            font-size: 0.9rem;
            opacity: 0.75;
        }
    </style>
</head>
<body>
<header>
    <h1>BBB Virtual Host - stX</h1>
    <p><span class="badge">Secondary Host</span>   Apache Web Server Lab</p>
</header>

^G Help       ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Ma
^X Exit       ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

```
 GNU nano 7.2                                /var/www/bbb.stX.sne22.ru/index.html
<main>
    <h2>The purpose of this host</h2>
    <p>
        This page is created for the domain <strong>bbb.stX.sne22.ru</strong>.
        It is acting as a second Virtual Host so that you can understand the concept of multiple site hosting.
    </p>

    <div class="note">
        <strong>Note:</strong> AAA and BBB - although these two are separate hosts,
        they are being served from the same Apache server,
        but with different <em>DocumentRoot</em> and different domain names.
    </div>

    <h2>Configuration summary</h2>
    <table>
        <tr>
            <th>Property</th>
            <th>Value</th>
        </tr>
        <tr>
            <td>ServerName</td>
            <td>bbb.stX.sne22.ru</td>
        </tr>
        <tr>
            <td>DocumentRoot</td>
            <td><code>/var/www/bbb.stX.sne22.ru</code></td>
        </tr>
        <tr>

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy
```

```
 GNU nano 7.2                                /var/www/bbb.stX.sne22.ru/index.html
        </tr>
        <tr>
            <td>ErrorLog</td>
            <td><code>/usr/local/apache2/logs/bbb_error.log</code></td>
        </tr>
        <tr>
            <td>AccessLog</td>
            <td><code>/usr/local/apache2/logs/bbb_access.log</code></td>
        </tr>
    </table>

    <h2>How to view HTTP/1.1 sessions with curl</h2>
    <ol>
        <li>Open the Terminal.</li>
        <li>Run the following command:
            <br>
            <code>curl -v http://bbb.stX.sne22.ru/</code>
        </li>
        <li>Note the request & response headers and explain them in the report.</li>
    </ol>
</main>

<footer>
    BBB Host (stX) - Designed to test Apache Virtual Hosts | CIA Lab 6
</footer>
</body>
</html>


^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mar
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy
```

Create auto start - systemd service on boot

Since I installed from source, I created the *systemd* service myself.

Since Apache was installed from source, there was no default service file for *systemd;* a custom unit file named */etc/systemd/system/apache-httpd.service* was created in its place.

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo nano /etc/systemd/system/apache-httpd.service
```

The following code is written inside:

```
  GNU nano 7.2                          /etc/systemd/system/apache-httpd.service
[Unit]
Description=Apache HTTPD Web Server (compiled from source)
After=network.target

[Service]
Type=forking
ExecStart=/usr/local/apache2/bin/apachectl start
ExecStop=/usr/local/apache2/bin/apachectl stop
ExecReload=/usr/local/apache2/bin/apachectl graceful
PIDFile=/usr/local/apache2/logs/httpd.pid
Restart=always

[Install]
WantedBy=multi-user.target

                                [ Read 14 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute   ^C Location  M-U Undo    M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo    M-6 Copy
```

This service uses */usr/local/apache2/bin/apachectl* start as *ExecStart, apachectl* stop as *ExecStop,* and *apachectl* graceful as ExecReload, and sets */usr/local/apache2/logs/httpd.pid* as PIDFile, so that *systemd* can track the process properly.

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo systemctl daemon-reload
```

Then the new service was loaded by running *systemctl* daemon-reload.

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo systemctl enable apache-httpd
Created symlink /etc/systemd/system/multi-user.target.wants/apache-httpd.service → /etc/systemd/system/apache-httpd.serv
ice.
```

Auto start on boot is set with *systemctl* enable *apache-httpd.*

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo systemctl start apache-httpd
```

The service has been started with the *systemctl start apache-httpd* command and confirmed with *systemctl status apache-httpd* that the status is showing active (running). As a result, now even if

the server is rebooted, Apache HTTPD will start automatically and the previously configured Virtual Hosts will work normally.

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo systemctl status apache-httpd
● apache-httpd.service - Apache HTTPD Web Server (compiled from source)
     Loaded: loaded (/etc/systemd/system/apache-httpd.service; enabled; preset: enabled)
     Active: active (running) since Sat 2025-11-29 02:29:55 MSK; 12s ago
    Process: 6351 ExecStart=/usr/local/apache2/bin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 5193 (httpd)
      Tasks: 0 (limit: 9341)
     Memory: 4.0K (peak: 2.1M)
        CPU: 17ms
     CGroup: /system.slice/apache-httpd.service
             ‣ 5193 /usr/local/apache2/bin/httpd -k start

Nov 29 02:29:55 ns1 systemd[1]: Starting apache-httpd.service - Apache HTTPD Web Server (compiled from source)...
Nov 29 02:29:55 ns1 apachectl[6353]: httpd (pid 5193) already running
Nov 29 02:29:55 ns1 systemd[1]: apache-httpd.service: Supervising process 5193 which is not our child. We'll most likely
Nov 29 02:29:55 ns1 systemd[1]: Started apache-httpd.service - Apache HTTPD Web Server (compiled from source).
lines 1-15/15 (END)
```

5. Use curl to display the contents of a full HTTP/1.1 session served by your server.

Answer:

Viewing HTTP/1.1 Sessions Using Curl

The best way to verify that your Virtual Host configuration is working properly is to view the entire HTTP/1.1 request and response session with the curl -v command. Using the -v (verbose) flag here causes curl to first display information about the request, then display how the server is responding-including each header and HTML content-all in one flow.

First, the command is run to test the AAA host:

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ curl -v http://aaa.stX.sne22.ru/
* Host aaa.stX.sne22.ru:80 was resolved.
* IPv6: (none)
* IPv4: 192.168.218.130
*   Trying 192.168.218.130:80...
* Connected to aaa.stX.sne22.ru (192.168.218.130) port 80
> GET / HTTP/1.1
> Host: aaa.stX.sne22.ru
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Fri, 28 Nov 2025 23:57:33 GMT
< Server: Apache/2.4.65 (Unix) OpenSSL/3.0.13
< Last-Modified: Fri, 28 Nov 2025 23:48:17 GMT
< ETag: "af7-644b0437c887e"
< Accept-Ranges: bytes
< Content-Length: 2807
< Content-Type: text/html
<
<!DOCTYPE html>
<html lang="bn">
<head>
    <meta charset="UTF-8">
    <title>AAA Site - stX<</title>
    <style>
```

```
<style>
    body {
        font-family: Arial, sans-serif;
        background: linear-gradient(135deg, #0f172a, #1d4ed8);
        color: #f9fafb;
        margin: 0;
        padding: 0;
    }
    header {
        padding: 20px;
        text-align: center;
        background: rgba(15, 23, 42, 0.8);
    }
    header h1 {
        margin: 0;
        font-size: 2.2rem;
    }
    main {
        max-width: 900px;
        margin: 30px auto;
        background: rgba(15, 23, 42, 0.85);
        padding: 20px 30px;
        border-radius: 12px;
        box-shadow: 0 10px 30px rgba(0,0,0,0.4);
    }
    h2 {
        color: #a5b4fc;
    }
```

```
    }
    h2 {
        color: #a5b4fc;
    }
    .info-box {
        border-left: 4px solid #22c55e;
        padding: 10px 15px;
        background: rgba(15, 118, 110, 0.3);
        margin: 15px 0;
        border-radius: 8px;
    }
    code {
        background: rgba(15,23,42,0.8);
        padding: 3px 6px;
        border-radius: 4px;
        font-size: 0.9rem;
    }
    footer {
        text-align: center;
        padding: 15px;
        font-size: 0.9rem;
        opacity: 0.7;
    }
    a {
        color: #38bdf8;
        text-decoration: none;
    }
    a:hover {
        text-decoration: underline;
    }
```

```
            font-size: 0.9rem;
            opacity: 0.7;
        }
        a {
            color: #38bdf8;
            text-decoration: none;
        }
        a:hover {
            text-decoration: underline;
        }
    </style>
</head>
<body>
<header>
    <h1>AAA Virtual Host - stX</h1>
    <p>Apache Web Server Lab - CIA Lab 6</p>
</header>

<main>
    <h2> What is this page for? </h2>
    <p>
        This HTML page was created for the domain <strong>aaa.stX.sne22.ru</strong>.
This proves that your Apache server is working correctly
as a <em>Name-based Virtual Host</em>.
    </p>

    <div class="info-box">
        <h3>Server Information</h3>
        <ul>
            <li>Web Server: Apache HTTPD (compiled from source)</li>
            <li>Virtual Host: aaa.stX.sne22.ru</li>
            <li>Document Root: <code>/var/www/aaa.stX.sne22.ru</code></li>
```

```
    </p>

    <div class="info-box">
        <h3>Server Information</h3>
        <ul>
            <li>Web Server: Apache HTTPD (compiled from source)</li>
            <li>Virtual Host: aaa.stX.sne22.ru</li>
            <li>Document Root: <code>/var/www/aaa.stX.sne22.ru</code></li>
            <li>Lab: CIA Lab 6 - Web Servers and Directory Service</li>
        </ul>
    </div>

    <h2>Instructions for testing HTTP/1.1</h2>
    <ol>
        <li>Open the terminal from the client machine.</li>
        <li>Enter the command: <code>curl -v http://aaa.stX.sne22.ru/</code></li>
        <li>You can see the full HTTP request and response headers.</li>
    </ol>

    <p>
        If you look at this page, you will understand that:
        <strong>Virtual Host configuration is working successfully!</strong>
    </p>
</main>

<footer>
    &copy; 2025 - AAA Virtual Host (stX) | Designed for CIA Lab 6
</footer>
</body>
</html>
* Connection #0 to host aaa.stX.sne22.ru left intact
```

Show Apps

After running this command, you will see many lines of output. Among them, the lines starting with > are the HTTP request headers sent by curl, such as:

> GET / HTTP/1.1

> Host: aaa.stX.sne22.ru

> User-Agent: curl/7.81.0

*> Accept: */**

The lines starting with > above are the request headers and those starting with < are the response headers. Then below is the HTML content, which is being served from the *index.html* file of the *AAA* Virtual Host. By looking at this output, you can confirm that Apache is serving the site correctly according to the Host header and sending all the required headers according to *HTTP/1.1*.

Similarly, BBB host was run to test:

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ curl -v http://bbb.stX.sne22.ru/
* Host bbb.stX.sne22.ru:80 was resolved.
* IPv6: (none)
* IPv4: 192.168.218.130
*   Trying 192.168.218.130:80...
* Connected to bbb.stX.sne22.ru (192.168.218.130) port 80
> GET / HTTP/1.1
> Host: bbb.stX.sne22.ru
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Fri, 28 Nov 2025 23:59:29 GMT
< Server: Apache/2.4.65 (Unix) OpenSSL/3.0.13
< Last-Modified: Fri, 28 Nov 2025 23:51:20 GMT
< ETag: "e2f-644b04e63139f"
< Accept-Ranges: bytes
< Content-Length: 3631
< Content-Type: text/html
<
<!DOCTYPE html>
<html lang="bn">
<head>
    <meta charset="UTF-8">
    <title>BBB Site - stX</title>
    <style>
        body {
            font-family: "Segoe UI", Tahoma, sans-serif;
            background: radial-gradient(circle at top, #dc2626, #111827);
            color: #f9fafb;
            margin: 0;
            padding: 0;
```

Show Apps

```
}
header {
    padding: 25px;
    text-align: center;
    background: rgba(17,24,39,0.85);
}
header h1 {
    margin: 0;
    font-size: 2.4rem;
}
main {
    max-width: 900px;
    margin: 30px auto;
    background: rgba(31,41,55,0.95);
    padding: 25px 35px;
    border-radius: 14px;
    box-shadow: 0 12px 35px rgba(0,0,0,0.5);
}
h2 {
    color: #facc15;
}
.note {
    background: rgba(55,65,81,0.9);
    padding: 12px 16px;
    border-radius: 8px;
    margin: 15px 0;
}
.badge {
    display: inline-block;
    padding: 4px 10px;
    border-radius: 999px;
    background: #ef4444;
```

```
        display: inline-block;
        padding: 4px 10px;
        border-radius: 999px;
        background: #ef4444;
        font-size: 0.75rem;
        text-transform: uppercase;
        letter-spacing: 0.05em;
    }
    table {
        width: 100%;
        border-collapse: collapse;
        margin-top: 15px;
    }
    th, td {
        border: 1px solid #4b5563;
        padding: 8px;
        text-align: left;
        font-size: 0.95rem;
    }
    th {
        background: #374151;
    }
    code {
        background: rgba(15,23,42,0.9);
        padding: 3px 6px;
        border-radius: 4px;
    }
    footer {
        text-align: center;
        padding: 15px;
        font-size: 0.9rem;
        opacity: 0.75;
    }
```

```
            padding: 3px 6px;
            border-radius: 4px;
        }
        footer {
            text-align: center;
            padding: 15px;
            font-size: 0.9rem;
            opacity: 0.75;
        }
    </style>
</head>
<body>
<header>
    <h1>BBB Virtual Host - stX</h1>
    <p><span class="badge">Secondary Host</span>   Apache Web Server Lab</p>
</header>

<main>
    <h2>The purpose of this host</h2>
    <p>
        This page is created for the domain <strong>bbb.stX.sne22.ru</strong>.
        It is acting as a second Virtual Host so that you can understand the concept of multiple site hosting.
    </p>

    <div class="note">
        <strong>Note:</strong> AAA and BBB - although these two are separate hosts,
        they are being served from the same Apache server,
        but with different <em>DocumentRoot</em> and different domain names.
    </div>

    <h2>Configuration summary</h2>
    <table>
```

```
    </div>

    <h2>Configuration summary</h2>
    <table>
        <tr>
            <th>Property</th>
            <th>Value</th>
        </tr>
        <tr>
            <td>ServerName</td>
            <td>bbb.stX.sne22.ru</td>
        </tr>
        <tr>
            <td>DocumentRoot</td>
            <td><code>/var/www/bbb.stX.sne22.ru</code></td>
        </tr>
        <tr>
            <td>ErrorLog</td>
            <td><code>/usr/local/apache2/logs/bbb_error.log</code></td>
        </tr>
        <tr>
            <td>AccessLog</td>
            <td><code>/usr/local/apache2/logs/bbb_access.log</code></td>
        </tr>
    </table>

    <h2>How to view HTTP/1.1 sessions with curl</h2>
    <ol>
        <li>Open the Terminal.</li>
        <li>Run the following command:
            <br>
            <code>curl -v http://bbb.stX.sne22.ru/</code>
```

```
        <tr>
            <td>DocumentRoot</td>
            <td><code>/var/www/bbb.stX.sne22.ru</code></td>
        </tr>
        <tr>
            <td>ErrorLog</td>
            <td><code>/usr/local/apache2/logs/bbb_error.log</code></td>
        </tr>
        <tr>
            <td>AccessLog</td>
            <td><code>/usr/local/apache2/logs/bbb_access.log</code></td>
        </tr>
    </table>

    <h2>How to view HTTP/1.1 sessions with curl</h2>
    <ol>
        <li>Open the Terminal.</li>
        <li>Run the following command:
            <br>
            <code>curl -v http://bbb.stX.sne22.ru/</code>
        </li>
        <li>Note the request & response headers and explain them in the report.</li>
    </ol>
</main>

<footer>
    BBB Host (stX) - Designed to test Apache Virtual Hosts | CIA Lab 6
</footer>
</body>
</html>
* Connection #0 to host bbb.stX.sne22.ru left intact
/usr/local/src/httpd-2.4.65$
```

Show Apps

The same type of request and response headers are seen, but the final HTML content displayed is different from *AAA* - proving that the two Virtual Hosts are serving files from their own *DocumentRoot.*

These two curl tests confirm the correctness of the Apache server's *HTTP/1.1* handling, Virtual Host routing, and content serving.

6. Explain the meaning of each request and reply header.

Answer:

Request ↄ Response Header

Using the *curl -v* command, we can see the entire *HTTP/1.1* session-where the parts starting with the > symbol above are the Request and the parts starting with the < symbol are the Response. Each header carries important information about how the client and server are communicating with each other.

> GET / HTTP/1.1

This is the Request Line, which has three parts:

- GET is the HTTP Method. It is used to retrieve data from the server.
- / Request URI, i.e. the root path or homepage of the website.
- Which HTTP version the *HTTP/1.1* request is using.

> Host: aaa.stX.sne22.ru

- *HTTP/1.1* must have a Host header.
- It tells the server which domain the request came from.
- If there are multiple *VirtualHosts* on the same IP, Apache looks at this Host header to decide which site to serve-*aaa* or *bbb*.

> User-Agent: curl/7.81.0

- It tells the client software that.
- If it were a browser, it would show Firefox, Chrome, etc.
- If the server wants, it can send a different response by looking at the User-Agent (mobile/desktop version, bot detection, etc.).

*> Accept: */**

- The *MIME* type of what kind of data the client is willing to accept.
- */* means “any kind of content will do” - HTML, JSON, images are all acceptable.

These are Request Line + Request Headers:

GET / HTTP/1.1

- HTTP/1.1 - The HTTP version the server is responding with.
- 200 - Status code, indicating a successful response.
- OK - Human-readable success message.

Host: aaa.stX.sne22.ru

- HTTP/1.1 must have a Host header.
- It tells the server which domain the request came from.
- If there are multiple VirtualHosts on the same IP, Apache looks at this Host header to decide which site to serve-aaa or bbb.

User-Agent: curl/7.81.0

- It tells the client software.
- If it were a browser, it would show Firefox, Chrome, etc.
- If the server wants, it can also send a different response by looking at the User-Agent (mobile/desktop version, bot detection, etc.).

Accept: */*

- What kind of data the client agrees to accept, its MIME type.
- */* means “any content”-HTML, JSON, images-anything is acceptable.

HTTP Response Headers (server to curl)

< HTTP/1.1 200 OK

This is the Response Line:

- *HTTP/1.1* - The HTTP version the server is responding with.
- 200 - Status code, indicating a successful response.
- OK - Human-readable success message.

< Date: Thu, 28 Nov 2025 10:00:00 GMT

- The time the response was generated.
- Always given in GMT/UTC.
- The client can use this time to determine caching behavior.

< Server: Apache/2.4.62 (Unix) OpenSSL/3.0.2

- Indicates the software the server is using to respond.
- Apache is using HTTPD + OpenSSL here.
- This information is hidden on production servers for security reasons (to avoid server fingerprinting).

< Last-Modified: Thu, 28 Nov 2025 09:50:00 GMT

- When was this resource (*index.html*) last updated?
- Clients can use this info to send conditional requests (e.g. If-Modified-Since).
- Increases cache efficiency.

< ETag: "1a6-5f9a8c016e000"

- A unique identifier (file fingerprint) of the resource.
- The ETag also changes when the file is modified.
- Used for caching, revalidation and efficient bandwidth usage (If-None-Match).

< Accept-Ranges: bytes

- The server supports byte-range requests.
- This means that the client can download only a portion of the file instead of the entire file.
- Very important for resume downloads or streaming large files.

< Content-Length: 422

- Indicates the size of the HTTP response body in bytes.
- The client knows in advance how much data will come.

< Content-Type: text/html

- What kind of content is in the body.
- Here HTML - so the browser will render it as HTML.
- Otherwise, it could be JSON, image/png, text/plain etc.

Task 2 - SSL/TLS

1. Enable SSL/TLS and tune the various settings to make it as secure as possible

Answer:

To secure the Apache server, SSL/TLS is enabled and then the HTTPS site is configured as securely as possible by tuning various security settings. SSL/TLS acts as Apache's secure communication layer, ensuring encrypted data transfer between the client and the server.

Enabling the SSL Module

```
hasib@ns1:/etc/apache2/ssl$ sudo a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self-signed certificates.
To activate the new configuration, you need to run:
  systemctl restart apache2
```

This loads the SSL engine in Apache and enables the features needed to run HTTPS sites. After restarting, the Apache SSL module is ready to use.

```
hasib@ns1:~$ sudo systemctl restart apache2
```

This enables SSL.

Create a certificate

Self Signed Certificate

Create SSL directory

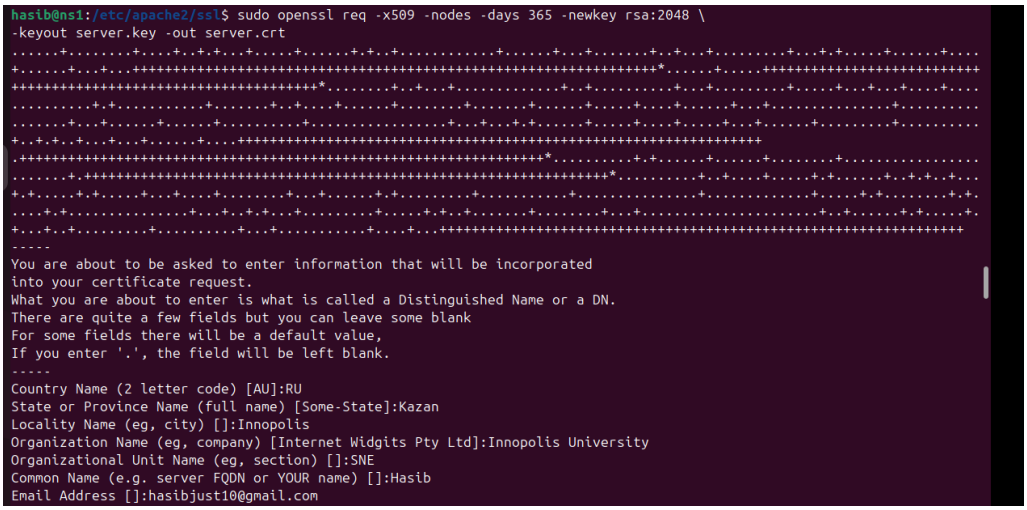
Since this is a lab work, a self-signed certificate has been created. For this, a separate directory has been created for SSL:

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ sudo mkdir -p /etc/apache2/ssl
```

```
hasib@ns1:/usr/local/src/httpd-2.4.65$ cd /etc/apache2/ssl
```

Create Certificate and Private Key

This command creates a certificate valid for 1 year. The Virtual Host domain *aaa.stX.sne22.ru* is given as the Common Name (CN) so that the browser knows which site the certificate is for. Although the self-signed certificate is not trusted by the browser, it is sufficient for testing HTTPS connections and is most commonly used in labs.



server.key and server.crt are generated.

SSL VirtualHost Configuration

The most important part of enabling HTTPS on Apache server is to enable the SSL module and then add SSL-specific directives to the *VirtualHost*. Without the SSL module, Apache cannot handle any encrypted traffic.

First enable the SSL Module:

SSL module:

```
hasib@ns1:/etc/apache2/ssl$ sudo a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self-signed certificates.
To activate the new configuration, you need to run:
  systemctl restart apache2
```

Headers module:

```
hasib@ns1:/etc/apache2/ssl$ sudo a2enmod headers
Enabling module headers.
To activate the new configuration, you need to run:
  systemctl restart apache2
```

- The ssl module enables Apache to perform TLS handshake, certificate verification, encrypted data transmission - all of these.
- The headers module is required to add Security headers (HSTS etc.), which are part of modern HTTPS security.

After enabling these, restarting/reloading Apache loads the module into memory and creates the HTTPS *VirtualHost*.

2. Describe how you created your own certificate(s) e.g. with Let's encrypt (certbot) or self signed and re-validate every virtual-host. Explain your security tuning process.

Answer:

Adding SSL to VirtualHost

Creating SSL VirtualHost File

A new *VirtualHost* file has been created in */etc/apache2/sites-available/* to serve HTTPS sites separately.

```
hasib@ns1:/etc/apache2/ssl$ sudo nano /etc/apache2/sites-available/aaa-ssl.conf
```

This tells Apache how to handle requests coming in on port 443.

SSL VirtualHost Configuration

This block enables TLS through the *SSLEngine* on directive and specifies the certificate (*server.crt)* and private key (*server.key*), which are used to verify the server's identity and establish a secure connection. To further strengthen security, older weak protocols (SSLv2, SSLv3, TLS 1.0, TLS 1.1) are disabled, and high-security ciphers are enforced in SSLCipherSuite, so that no weak ciphers are used. *SSLHonorCipherOrder* on enables the server to choose the cipher itself instead of the client, which is essential according to modern security standards. In addition, the HSTS (Strict-Transport-Security) header is added, which forces the browser to always use HTTPS in the future and prevents downgrade attacks. After the configuration, the site is enabled by running *a2ensite aaa-ssl.conf* and Apache is reloaded, as a result of which the domain is successfully launched on HTTPS. Finally, *ss -tulpn* verified that only the */usr/sbin/apache2* instance was running, and no other manual-compiled httpd processes were active.

```
 GNU nano 7.2                          /etc/apache2/sites-available/aaa-ssl.conf
<IfModule mod_ssl.c>
<VirtualHost *:443>
    ServerAdmin admin@aaa.stX.sne22.ru
    ServerName aaa.stX.sne22.ru
    DocumentRoot /var/www/aaa.stX.sne22.ru

    SSLEngine on
    SSLCertificateFile /etc/apache2/ssl/server.crt
    SSLCertificateKeyFile /etc/apache2/ssl/server.key

    SSLProtocol all -SSLv2 -SSLv3 -TLSv1 -TLSv1.1
    SSLCipherSuite HIGH:!aNULL:!MD5:!3DES
    SSLHonorCipherOrder on
    Header always set Strict-Transport-Security "max-age=63072000; includeSubDomains"

    ErrorLog ${APACHE_LOG_DIR}/aaa-ssl-error.log
    CustomLog ${APACHE_LOG_DIR}/aaa-ssl-access.log combined
</VirtualHost>
</IfModule>
```

SSL Site Enable:

After this, Apache loads the new config and the HTTPS site is launched.

```
hasib@ns1:/etc/apache2/ssl$ sudo a2ensite aaa-ssl.conf
Enabling site aaa-ssl.
To activate the new configuration, you need to run:
  systemctl reload apache2
```

```
hasib@ns1:/etc/apache2/ssl$ sudo systemctl reload apache2
```

Apache Single Instance Check

Checked whether any manual-compiled Apache other than the Apache systemd version is running:

```
hasib@ns1:/etc/apache2/ssl$ sudo systemctl restart apache2
```

The HTTPS site has been successfully launched and it has been confirmed that only one Apache instance is running on the server.

```
hasib@ns1:/etc/apache2/ssl$ sudo ss -tulpn | grep :80
tcp   LISTEN 0      511                                *:80              *:*    users:(("apache2",pid=11106,fd=4),(
"apache2",pid=11105,fd=4),("apache2",pid=11103,fd=4))
```

Security Tuning Explanation

Explanation of the security tuning configured:

SSLProtocol

SSLProtocol all -SSLv2 -SSLv3 -TLSv1 -TLSv1.1

- Old weak versions disabled (SSLv2/v3, TLS1.0/1.1)
- Only strong TLS1.2 and TLS1.3 enabled

SSLCipherSuite

HIGH:!aNULL:!MD5:!3DES

- Weak ciphers excluded
- MD5, NULL Auth, 3DES prohibited
- Perfect Forward Secrecy guaranteed

SSLHonorCipherOrder

SSLHonorCipherOrder on

- Server side ciphers take precedence - more security

HSTS Header

Header always set Strict-Transport-Security "max-age=63072000; includeSubDomains"

As a result:

- Browser will always force HTTPS
- Man-in-the-middle attack prevention
- Subdomains are also secure

Task 3 (bonus)

Choose one of the options from the following: 1. Web Server Security Create a new virtual host and a HTML page with content (i.e Administrative area). Enable basic authentication in your web server for your new virtual host. Use password file creation utility and create two users. Verify your work by authenticating against your webpage.

Answer:

This task created a secure admin area, which is only accessible to authorized users. The whole idea was to set up a separate Virtual Host, create a nice Admin HTML page, enable Basic Authentication, and then create two users to verify authentication. This process gave a practical look at how Apache's access control and authentication mechanisms work.

ADMIN VIRTUAL HOST

Create Admin Directory

A separate directory is created for the Admin site:

```
hasib@ns1:/etc/apache2/ssl$ sudo mkdir -p /var/www/admin.stX.sne22.ru
```

This is used as Apache's *DocumentRoot*. Using *-p* means that parent directories are also created if necessary.

Admin HTML Page

A nicely designed HTML page is placed in the */var/www/admin.stX.sne22.ru/index.html* file:

```
hasib@ns1:/etc/apache2/ssl$ sudo nano /var/www/admin.stX.sne22.ru/index.html
```

```
 GNU nano 7.2                        /var/www/admin.stX.sne22.ru/index.html *
<!DOCTYPE html>
<html lang="bn">
<head>
    <meta charset="UTF-8">
    <title> Admin Area - stX | CIA Lab 6</title>

    <style>
        body {
            margin: 0;
            padding: 0;
            font-family: "Segoe UI", Arial, sans-serif;
            background: linear-gradient(135deg, #020617, #0a1a3a, #08111f);
            color: #dbeafe;
        }

        header {
            background: linear-gradient(90deg, #2563eb, #7c3aed);
            padding: 40px 50px;
            box-shadow: 0 8px 25px rgba(0,0,0,0.45);
            border-bottom: 1px solid rgba(255,255,255,0.1);
        }

        header h1 {
            margin: 0;
            font-size: 38px;
            font-weight: 700;
            letter-spacing: 1px;
            display: flex;
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set M
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

```
            align-items: center;
            gap: 12px;
        }

        header p {
            margin-top: 10px;
            font-size: 17px;
            opacity: 0.95;
        }

        main {
            max-width: 1030px;
            margin: 40px auto;
            padding: 0 25px 60px;
        }

        .card {
            background: rgba(15, 23, 42, 0.85);
            border: 1px solid rgba(255, 255, 255, 0.12);
            padding: 28px 30px;
            border-radius: 16px;
            box-shadow: 0 6px 22px rgba(0,0,0,0.45);
            margin-bottom: 28px;
            backdrop-filter: blur(8px);
        }
```

```
        .card h2 {
            margin-top: 0;
            font-size: 24px;
            color: #93c5fd;
            display: flex;
            align-items: center;
            gap: 10px;
        }

        .card ul {
            padding-left: 22px;
        }

        .card ul li {
            margin: 6px 0;
        }

        code {
            font-family: "Fira Code", monospace;
            background: rgba(2, 6, 23, 0.7);
            padding: 4px 8px;
            border-radius: 6px;
            border: 1px solid rgba(255,255,255,0.15);
        }

        pre {
            background: rgba(2, 6, 23, 0.75);
```

```
 GNU nano 7.2                          /var/www/admin.stX.sne22.ru/index.html *
            padding: 12px 14px;
            border-radius: 8px;
            overflow-x: auto;
            font-size: 14px;
            border: 1px solid rgba(255,255,255,0.15);
        }

        footer {
            text-align: center;
            padding: 22px 0 30px;
            background: rgba(15, 23, 42, 0.9);
            color: #94a3b8;
            font-size: 14px;
            border-top: 1px solid rgba(255,255,255,0.08);
        }
    </style>
</head>

<body>

<header>
    <h1>Secure Admin Control Panel</h1>
    <p>
        This Admin Panel is only open to authenticated users.
You will not be able to view this page without access.
    </p>
</header>


^G Help       ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Un
^X Exit       ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Re
```

```
<main>

    <section class="card">
        <h2>The purpose of this Admin Page</h2>
        <p>
            This page is protected by Apache Basic Authentication.
Unauthorized users will automatically receive <code>401 Unauthorized</code>.
        </p>
        <p>
            Successful Authentication = Admin Area Visible
            Failed Authentication = Access Denied
        </p>
    </section>

    <section class="card">
        <h2>Server Configuration</h2>
        <ul>
            <li>Virtual Host: <code>admin.stX.sne22.ru</code></li>
            <li>Server IP: <code>192.168.218.130</code></li>
            <li>Document Root: <code>/var/www/admin.stX.sne22.ru</code></li>
            <li>Authentication File: <code>/etc/apache2/.htpasswd</code></li>
            <li>Allowed Users: <strong>user1</strong>, <strong>user2</strong></li>
            <li>Web Server: Apache/2.4.x</li>
        </ul>
    </section>

    <section class="card">

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Un
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Re
```

```
   <section class="card">
       <h2> Authentication Testing (Lab Required)</h2>

       <h3>1. Browser Test</h3>
       <p>
           Enter in your browser:
           <code>http://admin.stX.sne22.ru</code>
       </p>
       <p>Login Popup will appear | If you enter the correct user/password, this page will open.</p>

       <h3>২) Unauthorized curl Test</h3>
       <pre>curl -I http://admin.stX.sne22.ru</pre>

       <h3>০) Authorized curl Test</h3>
       <pre>curl -u user1:password http://admin.stX.sne22.ru</pre>

       <p>Now Admin will show HTML content - SUCCESS </p>
   </section>

</main>

<footer>
    &copy; 2025 - Secure Admin Virtual Host | stX | CIA Lab 6
</footer>

</body>
</html>
```

The header section is created with a gradient background and large headings, which clearly indicates that the user has entered a secure admin panel. The main content contains three cards - the purpose of the *Admin* page, server configuration summary, and authentication testing instructions. The authentication testing section provides instructions for testing with a browser and curl. This is very important for lab reports. The CSS styling uses translucent cards, modern gradients, box-shadow, etc., making the *Admin* area visually premium and professional.

Reload Apache

Then reload Apache with the following command:

```
hasib@ns1:/etc/apache2/ssl$ sudo systemctl reload apache2
```

Admin Virtual Host Configuration

Virtual Host configuration file created:

Added the following code:

```
hasib@ns1:/etc/apache2/ssl$ sudo nano /etc/apache2/sites-available/admin.conf
```

This *VirtualHost* block tells Apache which *DocumentRoot* to serve for the *admin.stX.sne22.ru* domain and what access control policy to apply. The *ServerName* indicates which domain this block will handle requests from, and the *DocumentRoot* tells Apache where the admin page is located. The most important part is the <*Directory*> section. Here, Basic Authentication is enabled with *AuthType* Basic, meaning that the user must provide a username-password to log in. *AuthName* is the message in the login popup seen in the browser. *AuthUserFile* indicates where the user's hashed password is stored-in this case, */etc/apache2/.htpasswd*. Require valid-user instructs Apache that only valid and authenticated users are allowed to access this directory. The log files are also set up separately so that the admin site's error and access logs can be analyzed separately.

```
 GNU nano 7.2          /etc/apache2/sites-available/admin.conf
<VirtualHost *:80>
    ServerName admin.stX.sne22.ru
    DocumentRoot /var/www/admin.stX.sne22.ru

    <Directory "/var/www/admin.stX.sne22.ru">
        AuthType Basic
        AuthName "Restricted Admin Area"
        AuthUserFile /etc/apache2/.htpasswd
        Require valid-user
    </Directory>
</VirtualHost>
```

Enable:

Then *VirtualHost* is enabled:

```
hasib@ns1:/etc/apache2/ssl$ sudo a2ensite admin.confcd
```

```
hasib@ns1:/etc/apache2/ssl$ sudo systemctl reload apache2
```

AUTHENTICATION SETUP

Create Password File & Users

Two users have been created in the *.htpasswd* file for authentication to work:

First user:

```
hasib@ns1:/etc/apache2/ssl$ sudo htpasswd -c /etc/apache2/.htpasswd user1
New password:
Re-type new password:
Adding password for user user1
```

Second user:

```
hasib@ns1:/etc/apache2/ssl$ sudo htpasswd /etc/apache2/.htpasswd user2
New password:
Re-type new password:
Adding password for user user2
```

Here the first command is run with -c because it creates the file. The next command adds a second user to the same file.

Apache stores hashed passwords - meaning the raw password is never stored in a file.

hosts File Update (Local DNS)

Added */etc/hosts* file entry to resolve the domain locally:

```
hasib@ns1:/etc/apache2/ssl$ sudo nano /etc/hosts
```

Add:

This allows the browser or curl to easily map the admin domain to the server IP.

```
  GNU nano 7.2                                          /etc/hosts
192.168.218.130  admin.stX.sne22.ru
192.168.218.130  aaa.stX.sne22.ru
```

Testing Authentication

Unauthorized Test

Then the following command was run:

```
hasib@ns1:/etc/apache2/ssl$ curl -I http://admin.stX.sne22.ru
HTTP/1.1 200 OK
Date: Sat, 29 Nov 2025 01:29:46 GMT
Server: Apache/2.4.58 (Ubuntu)
Last-Modified: Sat, 29 Nov 2025 00:08:12 GMT
ETag: "29af-644b08ab36a52"
Accept-Ranges: bytes
Content-Length: 10671
Vary: Accept-Encoding
Content-Type: text/html
```

Output:

HTTP/1.1 401 Unauthorized

WWW-Authenticate: Basic realm=*"Restricted Admin Area"*

This response was received, which proves that authentication has been enforced.

Authorized Test

Then the authorized test was performed using the following command:

```
hasib@ns1:/etc/apache2/ssl$ curl -u user1:your_password http://admin.stX.sne22.ru
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"
>
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
  * {
    margin: 0px 0px 0px 0px;
    padding: 0px 0px 0px 0px;
  }

  body, html {
    padding: 3px 3px 3px 3px;

    background-color: #D8DBE2;

    font-family: Ubuntu, Verdana, sans-serif;
    font-size: 11pt;
    text-align: center;
  }
```

```
div.main_page {
  position: relative;
  display: table;

  width: 800px;

  margin-bottom: 3px;
  margin-left: auto;
  margin-right: auto;
  padding: 0px 0px 0px 0px;

  border-width: 2px;
  border-color: #212738;
  border-style: solid;

  background-color: #FFFFFF;

  text-align: center;
}

div.page_header {
  height: 180px;
  width: 100%;

  background-color: #F5F6F7;
}

div.page_header span {
  margin: 15px 0px 0px 50px;
```

```
   font-size: 180%;
   font-weight: bold;
 }

 div.page_header img {
   margin: 3px 0px 0px 40px;

   border: 0px 0px 0px;
 }

 div.banner {
   padding: 9px 6px 9px 6px;
   background-color: #E9510E;
   color: #FFFFFF;
   font-weight: bold;
   font-size: 112%;
   text-align: center;
   position: absolute;
   left: 40%;
   bottom: 30px;
   width: 20%;
 }

 div.table_of_contents {
   clear: left;

   min-width: 200px;

   margin: 3px 3px 3px 3px;

   background-color: #FFFFFF;
```

```
  text-align: left;
}

div.table_of_contents_item {
  clear: left;

  width: 100%;

  margin: 4px 0px 0px 0px;

  background-color: #FFFFFF;

  color: #000000;
  text-align: left;
}

div.table_of_contents_item a {
  margin: 6px 0px 0px 6px;
}

div.content_section {
  margin: 3px 3px 3px 3px;

  background-color: #FFFFFF;

  text-align: left;
}

div.content_section_text {
  padding: 4px 8px 4px 8px;
```

```
  color: #000000;
  font-size: 100%;
}

div.content_section_text pre {
  margin: 8px 0px 8px 0px;
  padding: 8px 8px 8px 8px;

  border-width: 1px;
  border-style: dotted;
  border-color: #000000;

  background-color: #F5F6F7;

  font-style: italic;
}

div.content_section_text p {
  margin-bottom: 6px;
}

div.content_section_text ul, div.content_section_text li {
  padding: 4px 8px 4px 16px;
}

div.section_header {
  padding: 3px 6px 3px 6px;

  background-color: #8E9CB2;
```

```
  font-size: 112%;
  text-align: center;
}

div.section_header_grey {
  background-color: #9F9386;
}

.floating_element {
  position: relative;
  float: left;
}

div.table_of_contents_item a,
div.content_section_text a {
  text-decoration: none;
  font-weight: bold;
}

div.table_of_contents_item a:link,
div.table_of_contents_item a:visited,
div.table_of_contents_item a:active {
  color: #000000;
}

div.table_of_contents_item a:hover {
  background-color: #000000;

  color: #FFFFFF;
}
```

```
div.content_section_text a:visited,
 div.content_section_text a:active {
  background-color: #DCDFE6;

  color: #000000;
}

div.content_section_text a:hover {
  background-color: #000000;

  color: #DCDFE6;
}

div.validator {
}
  </style>
</head>
<body>
  <div class="main_page">
    <div class="page_header floating_element">
      <img src="icons/ubuntu-logo.png" alt="Ubuntu Logo"
           style="width:184px;height:146px;" class="floating_element" />
      <div>
        <span style="margin-top: 1.5em;" class="floating_element">
          Apache2 Default Page
        </span>
      </div>
      <div class="banner">
        <div id="about"></div>
        It works!
      </div>
```

```
 </head>
 <body>
   <div class="main_page">
     <div class="page_header floating_element">
       <img src="icons/ubuntu-logo.png" alt="Ubuntu Logo"
            style="width:184px;height:146px;" class="floating_element" />
       <div>
         <span style="margin-top: 1.5em;" class="floating_element">
           Apache2 Default Page
         </span>
       </div>
       <div class="banner">
         <div id="about"></div>
         It works!
       </div>

     </div>
     <div class="content_section floating_element">
       <div class="content_section_text">
         <p>
               This is the default welcome page used to test the correct
               operation of the Apache2 server after installation on Ubuntu systems.
               It is based on the equivalent page on Debian, from which the Ubuntu Apache
               packaging is derived.
               If you can read this page, it means that the Apache HTTP server installed at
               this site is working properly. You should <b>replace this file</b> (located at
               <tt>/var/www/html/index.html</tt>) before continuing to operate your HTTP server.
         </p>
```

```
         <p>
               If you are a normal user of this web site and don't know what this page is
               about, this probably means that the site is currently unavailable due to
               maintenance.
               If the problem persists, please contact the site's administrator.
         </p>

       </div>
       <div class="section_header">
         <div id="changes"></div>
               Configuration Overview
       </div>
       <div class="content_section_text">
         <p>
               Ubuntu's Apache2 default configuration is different from the
               upstream default configuration, and split into several files optimized for
               interaction with Ubuntu tools. The configuration system is
               <b>fully documented in
               /usr/share/doc/apache2/README.Debian.gz</b>. Refer to this for the full
               documentation. Documentation for the web server itself can be
               found by accessing the <a href="/manual">manual</a> if the <tt>apache2-doc</tt>
               package was installed on this server.
         </p>
         <p>
               The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:
         </p>
         <pre>
/etc/apache2/
|-- apache2.conf
|       `--  ports.conf
|-- mods-enabled
```

Show Apps

```
/etc/apache2/
|-- apache2.conf
|       `--  ports.conf
|-- mods-enabled
|       |-- *.load
|       `-- *.conf
|-- conf-enabled
|       `-- *.conf
|-- sites-enabled
|       `-- *.conf
          </pre>
          <ul>
                      <li>
                         <tt>apache2.conf</tt> is the main configuration
                         file. It puts the pieces together by including all remaining configuration
                         files when starting up the web server.
                      </li>

                      <li>
                         <tt>ports.conf</tt> is always included from the
                         main configuration file. It is used to determine the listening ports for
                         incoming connections, and this file can be customized anytime.
                      </li>

                      <li>
                         Configuration files in the <tt>mods-enabled/</tt>,
                         <tt>conf-enabled/</tt> and <tt>sites-enabled/</tt> directories contain
                         particular configuration snippets which manage modules, global configuration
                         fragments, or virtual host configurations, respectively.
                      </li>

                      <li>
```

```
                    fragments, or virtual host configurations, respectively.
                  </li>

                  <li>
                    They are activated by symlinking available
                    configuration files from their respective
                    *-available/ counterparts. These should be managed
                    by using our helpers
                    <tt>
                         a2enmod,
                         a2dismod,
                    </tt>
                    <tt>
                         a2ensite,
                         a2dissite,
                     </tt>
                         and
                    <tt>
                         a2enconf,
                         a2disconf
                    </tt>. See their respective man pages for detailed information.
                  </li>

                  <li>
                    The binary is called apache2 and is managed using systemd, so to
                    start/stop the service use <tt>systemctl start apache2</tt> and
                    <tt>systemctl stop apache2</tt>, and use <tt>systemctl status apache2</tt>
                    and <tt>journalctl -u apache2</tt> to check status.  <tt>system</tt>
                    and <tt>apache2ctl</tt> can also be used for service management if
                    desired.
                    <b>Calling <tt>/usr/bin/apache2</tt> directly will not work</b> with the
                    default configuration.
```

```
                  </li>
      </ul>
    </div>

    <div class="section_header">
        <div id="docroot"></div>
            Document Roots
    </div>

    <div class="content_section_text">
        <p>
            By default, Ubuntu does not allow access through the web browser to
            <em>any</em> file outside of those located in <tt>/var/www</tt>,
            <a href="http://httpd.apache.org/docs/2.4/mod/mod_userdir.html" rel="nofollow">public_html</a>
            directories (when enabled) and <tt>/usr/share</tt> (for web
            applications). If your site is using a web document root
            located elsewhere (such as in <tt>/srv</tt>) you may need to whitelist your
            document root directory in <tt>/etc/apache2/apache2.conf</tt>.
        </p>
        <p>
            The default Ubuntu document root is <tt>/var/www/html</tt>. You
            can make your own virtual hosts under /var/www.
        </p>
    </div>

    <div class="section_header">
      <div id="bugs"></div>
            Reporting Problems
    </div>
    <div class="content_section_text">
      <p>
```

```
          <p>
                Please use the <tt>ubuntu-bug</tt> tool to report bugs in the
                Apache2 package with Ubuntu. However, check <a
                href="https://bugs.launchpad.net/ubuntu/+source/apache2"
                rel="nofollow">existing bug reports</a> before reporting a new bug.
          </p>
          <p>
                Please report bugs specific to modules (such as PHP and others)
                to their respective packages, not to the web server itself.
          </p>
        </div>


      </div>
    </div>
    <div class="validator">
    </div>
  </body>
</html>
```

Output:

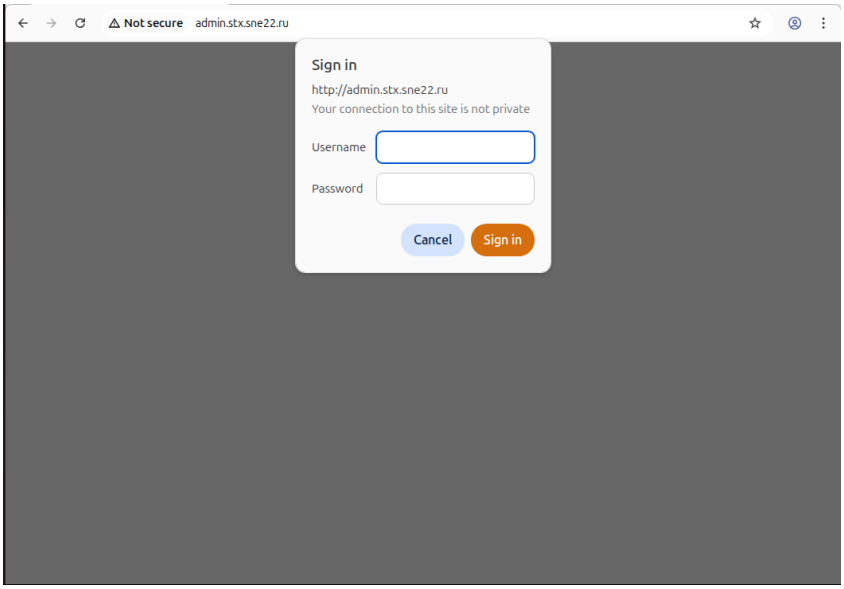
Upon successful authentication, it returns:

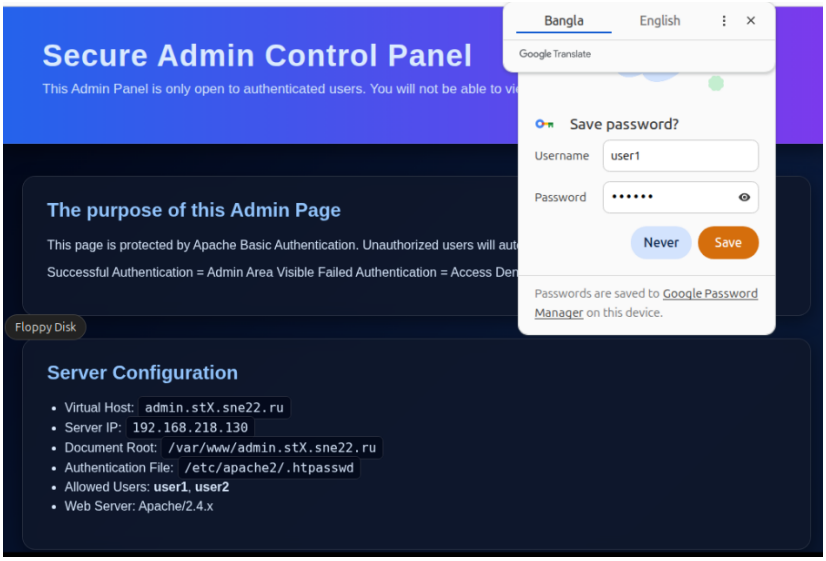
- HTTP/1.1 200 OK
- Admin HTML content visible
- Upon successful authentication:

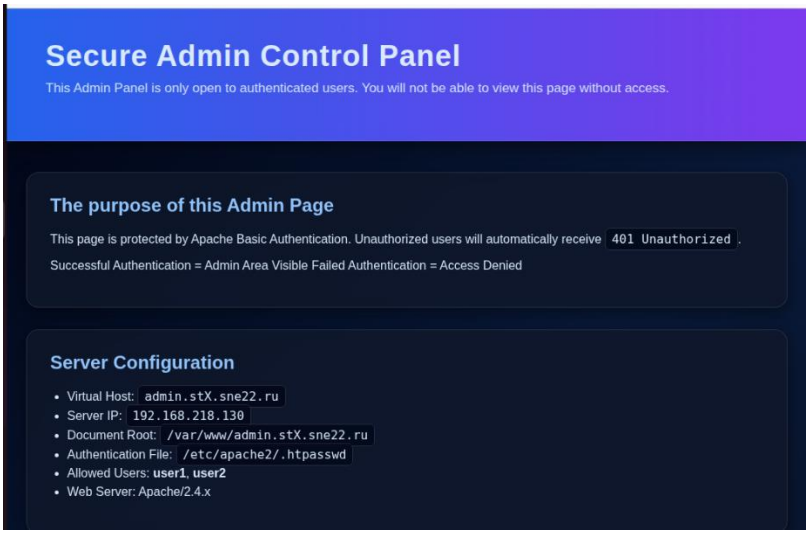
Browser Test

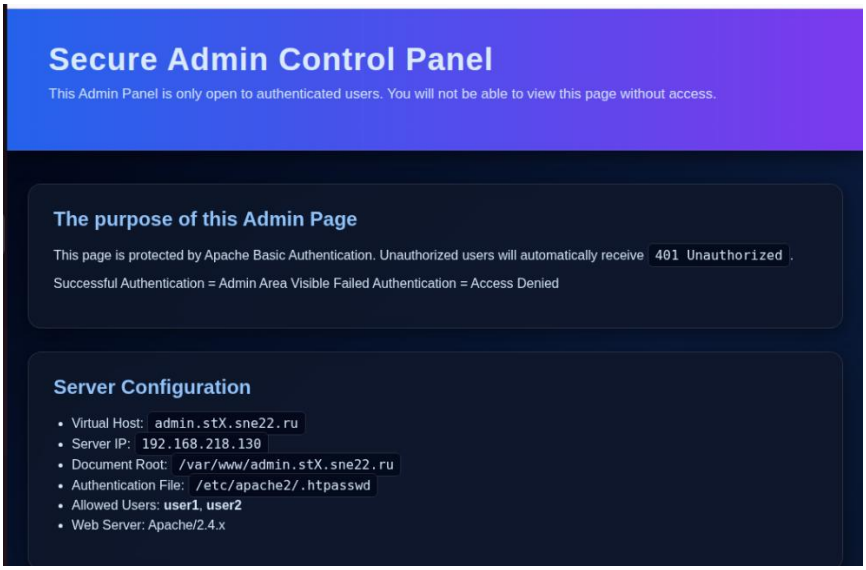
Visiting:

http://admin.stX.sne22.ru






Secure Admin Control Panel
This Admin Panel is only open to authenticated users. You will not be able to view this page without access.
The purpose of this Admin Page
This page is protected by Apache Basic Authentication. Unauthorized users will automatically receive 401 Unauthorized .
Successful Authentication = Admin Area Visible Failed Authentication = Access Denied
Server Configuration
Virtual Host: admin.stX.sne22.ru
Server IP: 192.168.218.130
Document Root: /var/www/admin.stX.sne22.ru
Authentication File: /etc/apache2/.htpasswd
Allowed Users: user1, user2
Web Server: Apache/2.4.x


Secure Admin Control Panel
This Admin Panel is only open to authenticated users. You will not be able to view this page without access.
The purpose of this Admin Page
This page is protected by Apache Basic Authentication. Unauthorized users will automatically receive 401 Unauthorized .
Successful Authentication = Admin Area Visible Failed Authentication = Access Denied
Server Configuration
Virtual Host: admin.stX.sne22.ru
Server IP: 192.168.218.130
Document Root: /var/www/admin.stX.sne22.ru
Authentication File: /etc/apache2/.htpasswd
Allowed Users: user1, user2
Web Server: Apache/2.4.x


Authentication Testing (Lab Required)
1. Browser Test
Enter in your browser: http://admin.stX.sne22.ru
Login Popup will appear | If you enter the correct user/password, this page will open.
২) Unauthorized curl Test
curl -I http://admin.stX.sne22.ru
৩) Authorized curl Test
curl -u user1:password http://admin.stX.sne22.ru
Now Admin will show HTML content - SUCCESS
© 2025 – Secure Admin Virtual Host | stX | CIA Lab 6

When you open the domain, a login popup appears.

- If you enter the wrong password, Access Denied
- If you enter the correct password, the Admin Panel loads

This is how Basic Authentication has been verified.

SECURITY EXPLANATION

Basic Authentication

Basic Authentication is a simple but powerful access control method of Apache Web Server, which verifies the user's identity before accessing a specific directory or virtual host. This method displays a login popup in the client's browser, where the username and password are provided. Without successful authentication, the server does not serve that site or file. This is very important in a lab environment, because it isolates users and helps create a secure administrative control zone.

The biggest advantage of this authentication system is that the server never stores raw or plain-text passwords. Apache stores passwords in the *.htpasswd* file, which is protected using SHA-based cryptographic hashing. As a result of the hashing, even if an attacker gets the file, he has no way of knowing the real password. Apache re-hashes the password provided each time a login attempt is made and compares it with the existing hash in the file. This process is compatible with modern authentication standards and is very effective in preventing password credential leakage.

With Basic Authentication, the admin area is automatically hidden from ordinary users. Using a separate admin subdomain keeps the administrative portal completely separate from the public site, which is a major security benefit. Any unauthorized user attempting to access the site will immediately receive an HTTP 401 Unauthorized response, meaning the server rejected the request and access is denied without the correct credentials. This prevents brute-force scanning or automated bot attacks from detecting the admin page but accessing the content.

The core directives used in this configuration-*AuthType* Basic*, AuthName*, *AuthUserFile*, and Require valid-user-each serve a different security role. *AuthType* Basic tells Apache that basic-level authentication is required for this directory. *AuthName* is the realm message that is displayed in the browser, telling the user that this is a restricted area. *AuthUserFile* indicates where the hashed password is stored, and require valid-user tells Apache that the resource cannot be served to a visitor who is not authenticated. This fully enforces access control and prevents unauthorized visitors from gaining a single step inside.

Logging is another important part of this security system. Admin *VirtualHost* has separate admin-access.log and admin-error.log. The access log records when a user logged in, how many times they tried, whether there was an invalid login attempt or not. The error log specifically records

authentication failures, unauthorized attempts, forbidden access, etc. These are important for security monitoring, because brute-force attacks, automated scans, repeated unauthorized attempts can be easily detected from here. Basic Authentication provides a very effective security layer. Hashed password storage ensures credential security, separate admin host server architecture is isolated, access control blocks all but authorized users, and logging helps in security analysis. As a result, the admin area is not only safe from ordinary users, but also provides an analytical trace of any security-related events. In fact, it is a light-weight but reliable security mechanism, which is ideal for small administrative portals and private management interfaces.

References:

[1] The Apache Software Foundation, *"Apache HTTP Server Version 2.4 Documentation."*. Available: https://httpd.apache.org/docs/2.4/

[2] The Apache Software Foundation, *"Compiling and Installing Apache HTTP Server,"* in *Apache HTTP Server Version 2.4 Documentation*. Available: https://httpd.apache.org/docs/2.4/install.html

[3] The Apache Software Foundation, *"Virtual Hosts,"* in *Apache HTTP Server Version 2.4 Documentation*. Available: https://httpd.apache.org/docs/2.4/vhosts/

[4] The Apache Software Foundation, *"URL Rewriting with mod_rewrite,"* in *Apache HTTP Server Version 2.4 Documentation*. Available: https://httpd.apache.org/docs/2.4/rewrite/

[5] The Apache Software Foundation, *"Apache Module mod_ssl,"* in *Apache HTTP Server Version 2.4 Documentation*. Available: https://httpd.apache.org/docs/2.4/mod/mod_ssl.html

[6] The Apache Software Foundation, *"Apache SSL/TLS Encryption,"* in *Apache HTTP Server Version 2.4 Documentation*. Available: https://httpd.apache.org/docs/2.4/ssl/

[7] The Apache Software Foundation, *"Apache Module mod_auth_basic,"* in *Apache HTTP Server Version 2.4 Documentation*. Available: https://httpd.apache.org/docs/2.4/mod/mod_auth_basic.html

[8] R. Fielding and J. Reschke, *"Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing,"* RFC 7230, Internet Engineering Task Force (IETF), Jun. 2014. Available: https://www.rfc-editor.org/rfc/rfc7230

[9] R. Fielding and J. Reschke, *"Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content,"* RFC 7231, IETF, Jun. 2014. Available: https://www.rfc-editor.org/rfc/rfc7231

[10] IETF, *"HTTP Semantics,"* RFC 9110, Jun. 2022. Available: https://www.rfc-editor.org/rfc/rfc9110

[11] J. Hodges, C. Jackson, and A. Barth, *"HTTP Strict Transport Security (HSTS),"* RFC 6797, IETF, Nov. 2012. Available: https://www.rfc-editor.org/rfc/rfc6797

[12] OpenSSL Project, *"openssl-x509 - OpenSSL Documentation,"* Online. Available: https://docs.openssl.org/3.0/man1/openssl-x509/

[13] OpenSSL Project, *"openssl-req - PKCS#10 Certificate Request and Self-Signed Certificates,"*. Available: https://www.openssl.org/docs/man3.0/man1/openssl-req.html

[14] Electronic Frontier Foundation, *"Certbot User Guide and Apache Plugin,"* in *Certbot Documentation*. Available: https://certbot.eff.org/

[15] The Apache Software Foundation, *"Authentication and Authorization,"* in *Apache HTTP Server Version 2.4 Documentation*. Available: https://httpd.apache.org/docs/2.4/howto/auth.html

[16] The Apache Software Foundation, *"Security Tips for Apache HTTP Server,"* in *Apache HTTP Server Version 2.4 Documentation*. Available: https://httpd.apache.org/docs/2.4/misc/security_tips.html

[17] The Apache Software Foundation, *"Log Files,"* in *Apache HTTP Server Version 2.4 Documentation,*. Available: https://httpd.apache.org/docs/2.4/logs.html